Sistema Agregador de Ferrocarriles Españoles

Hito 3 | Grupo 4

Víctor Elvira Fernández, Juan Horrillo Crespo, Sergio Velasco de Pedro

29-04-2026

Contents

1 Tren-ES **2**
1.1 Supuesto elegido . . . 2
1.2 Esquema mediador . . . 2
1.2.1 Tabla Estacion . . . 2
1.2.2 Tabla Municipio . . . 2
1.2.3 Tabla Viaje . . . 3
1.2.4 Tabla Parada . . . 3
1.2.5 Tabla Ruta . . . 3
1.2.6 Tabla Distancia . . . 3
1.3 Diagrama entidad-relación del mediador . . . 4
1.4 Formulación SQL de las consultas . . . 4
1.5 Captura de una instancia . . . 4

2 Esquemas Origen **9**
2.1 Forma conjuntiva de las tablas origen . . . 10
2.1.1 Municipio . . . 12
2.1.2 Estacion . . . 12
2.1.3 Distancia . . . 12
2.1.4 Ruta y Parada . . . 12
2.1.5 Viaje . . . 13
2.1.6 wikidata_MUNICIPIO . . . 13
2.1.7 INE_MUNICIPIO . . . 13
2.1.8 INE_PROVINCIA . . . 14
2.1.9 data_renfe_ESTACION . . . 14
2.1.10 horarios_renfe_RUTA . . . 14
2.1.11 adif_SALIDAS . . . 14

3 Reformulación GAV y LAV de las Consultas **14**
3.1 Consulta 1 . . . 14
3.1.1 Reformulación GAV . . . 14
3.1.2 Reformulación LAV . . . 15
3.2 Consulta 2 . . . 16
3.2.1 Reformulación GAV/LAV . . . 16
3.3 Consulta 3 . . . 17

4 Scraping **17**
4.1 ADIF . . . 17
4.2 RENFE . . . 18

5 SPARQL **20**
5.1 WikiData 20

1 Tren-ES

1.1 Supuesto elegido

Hemos elegido realizar la práctica sobre un supuesto propio, distinto de los propuestos. El objetivo consiste en construir un agregador que integre información acerca de la situación ferroviaria nacional. El sistema relacionará información sobre las estaciones, rutas y viajes en tren por el territorio español. El objetivo final será realizar las siguientes consultas:

1. Estaciones de tren en poblaciones de menos de 10000 habitantes.
2. Viajes entre dos estaciones a menos de 30 km entre sí.
3. Poblaciones de entre 20000 y 100000 habitantes en las que haya menos de cinco viajes programados hoy.

1.2 Esquema mediador

Para modelar la información del sistema se ha definido un esquema mediador compuesto por seis relaciones principales: **Estacion**, **Municipio**, **Viaje**, **Parada**, **Ruta** y **Distancia**. Este esquema permite representar tanto la estructura geográfica del sistema ferroviario como la planificación de trayectos y viajes.

Las relaciones del esquema mediador son las siguientes:

- Estacion(**id**, nombre, *municipio*, latitud, longitud)
- Municipio(nombre, n_habitantes, **id**, latitud, longitud, provincia, CCAA)
- Viaje(**id**, *ruta*, fecha, horario)
- Parada(***ruta***, ***estacion***, n_secuencia, km_origen)
- Ruta(**id**, *origen*, *destino*, tipo)
- Distancia(***estacion1***, ***estacion2***, distancia)

A continuación se describe brevemente cada una de las tablas y los atributos que la componen.

1.2.1 Tabla Estacion

La tabla **Estacion** guarda la información de las estaciones de tren registradas en el sistema. Cada estación está asociada a un municipio concreto.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de forma única cada estación.
- **nombre**: nombre de la estación.
- **municipio**: clave foránea que referencia al atributo **id** de la tabla **Municipio**. Indica el municipio en el que se encuentra situada la estación.
- **latitud**: coordenada geográfica de latitud de la estación.
- **longitud**: coordenada geográfica de longitud de la estación.

1.2.2 Tabla Municipio

La tabla **Municipio** almacena la información básica de cada municipio incluido en el sistema. En ella se recogen tanto datos identificativos como datos demográficos y geográficos.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de manera única cada municipio.
- **nombre**: nombre del municipio.
- **n_habitantes**: número de habitantes del municipio.
- **latitud**: coordenada geográfica de latitud del municipio.
- **longitud**: coordenada geográfica de longitud del municipio.

- **provincia**: provincia a la que pertenece el municipio.
- **CCAA**: comunidad autónoma a la que pertenece el municipio.

1.2.3 Tabla Viaje

La tabla **Viaje** almacena los viajes concretos programados en el sistema. Mientras que una ruta define un recorrido general, un viaje representa una realización específica de dicha ruta en una fecha y hora determinadas.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de forma única cada viaje.
- **ruta**: clave foránea que referencia al atributo **id** de la tabla **Ruta**. Indica qué ruta realiza el viaje.
- **fecha**: fecha en la que está programado el viaje.
- **horario**: hora programada para la salida del viaje.

1.2.4 Tabla Parada

La tabla **Parada** almacena las estaciones por las que pasa una ruta, guardando el orden de llegada planificado en la ruta a la que pertenece. Esta tabla NO tiene en cuenta lla existencia de rutas circulares (que por otra parte creemos que no existen en recorridos de alta y media distancia). Consideramos que las rutas tienen un origen y un destino, con una serie de paradas intermedias.

Sus atributos son:

- **ruta**: clave foránea que referencia al atributo **id** de la tabla **Ruta**. Forma parte de la clave primaria compuesta de la tabla.
- **estacion**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Forma parte de la clave primaria compuesta de la tabla.
- **n_secuencia**: indica el orden de la parada dentro de la ruta.
- **km_origen**: distancia en kilómetros desde el origen de la ruta hasta esa parada.

La clave primaria de **Parada** es compuesta, formada por los atributos **ruta** y **estacion**.

1.2.5 Tabla Ruta

La tabla **Ruta** representa los recorridos ferroviarios entre estaciones. Cada ruta conecta una estación de origen con una estación de destino y permite clasificar el trayecto según su tipo.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de manera única cada ruta.
- **origen**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Indica la estación de origen de la ruta.
- **destino**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Indica la estación de destino de la ruta.
- **tipo**: tipo de ruta o servicio ferroviario asociado.

1.2.6 Tabla Distancia

La tabla **Distancia** almacena la distancia entre pares de estaciones. Esta relación se utiliza para facilitar consultas sobre proximidad geográfica entre estaciones.

Sus atributos son:

- **estacion1**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Forma parte de la clave primaria compuesta.
- **estacion2**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Forma parte de la clave primaria compuesta.
- **distancia**: distancia entre ambas estaciones.

La clave primaria de **Distancia** es compuesta, formada por **estacion1** y **estacion2**.

1.3 Diagrama entidad-relación del mediador

1.4 Formulación SQL de las consultas

Podemos formular las consultas de la siguiente manera

1. **Estaciones de tren en poblaciones de menos de 10000 habitantes.**

```sql
SELECT E.id, E.nombre AS estacion, M.nombre AS municipio
    FROM Estacion AS E
    INNER JOIN Municipio AS M
        ON E.municipio = M.id
    WHERE M.n_habitantes < 10000;
```

2. **Viajes entre dos estaciones a menos de 30 km entre sí.**

Para la solución a esta consulta es importante ver a qué nos referimos con *distancia*: es la distancia euclídea directa entre las estaciones origen y destino a partir de su longitud y latitud. No tiene en cuenta el recorrido que haga, pudiendo ser este mayor que 30 km.

```sql
SELECT V.id
    FROM Viaje AS V
    INNER JOIN Ruta AS R
        ON V.ruta = R.id
    INNER JOIN Distancia AS D
        ON (R.origen = D.estacion1 AND R.destino = D.estacion2) OR
        (R.origen = D.estacion2 AND R.destino = D.estacion1)
    WHERE D.distancia < 30;
```

3. **Poblaciones de entre 20000 y 100000 habitantes en las que haya entre uno y cinco viajes programados hoy.**

Esta consulta en principio iba a ser **Poblaciones ... haya menos de cinco viajes programados hoy. El problema con esta consulta así formulada es que es necesario hacer un *OUTER JOIN* para obtener aquellas poblaciones en las que haya 0 viajes programados, lo que daba problemas al poner las consultas en forma conjuntiva, por lo que hemos decidido realizar esta simplificación.

```sql
SELECT
    M.id,
    M.nombre,
    COUNT(DISTINCT V.id) AS num_viajes_hoy
FROM
    Municipio AS M
    INNER JOIN Estacion AS E ON E.municipio_id = M.id
    INNER JOIN Parada AS P ON P.estacion = E.id
    INNER JOIN Viaje AS V ON V.ruta = P.ruta
WHERE
    M.n_habitantes BETWEEN 20000 AND 100000
    AND V.fecha = CURRENT_DATE
GROUP BY
    M.id,
    M.nombre
HAVING
    COUNT(DISTINCT V.id) BETWEEN 1 AND 3;
```

1.5 Captura de una instancia

En este apartado se muestran las capturas de instancias específicas del esquema mediador:

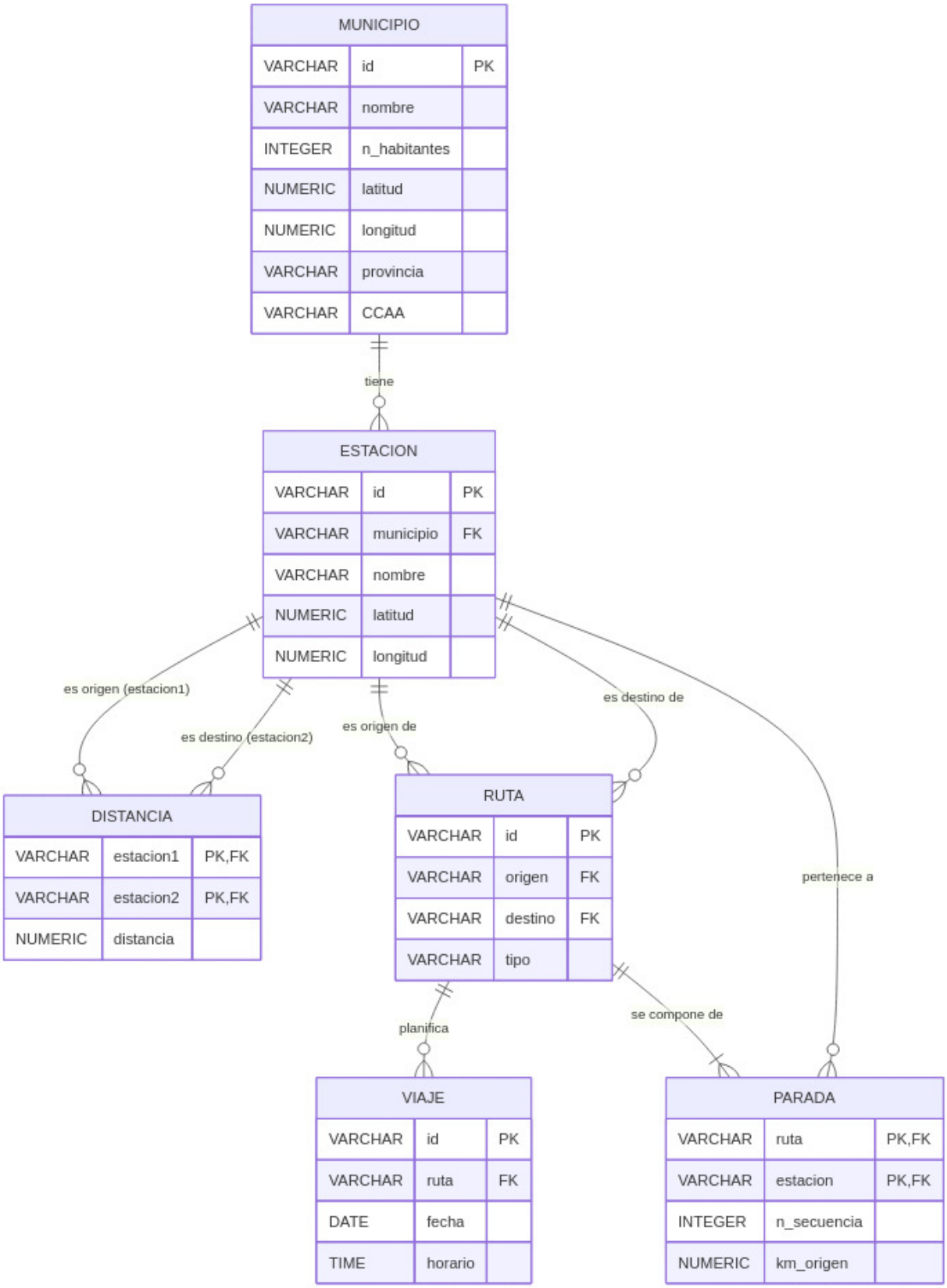


Figure 1: Esquema del mediador

Ruta

id	origen	destino	tipo
1	1	10	AVE
2	10	1	AVE
3	1	2	AVANT
4	2	10	AVE
5	1	3	Media Distancia
6	3	5	Media Distancia
7	6	7	Regional
8	8	10	Media Distancia
9	4	1	Media Distancia
10	9	10	Alvia
11	5	3	Media Distancia
12	7	6	Regional
13	10	8	Media Distancia
14	1	4	Media Distancia
15	10	9	Alvia

Distancia

estacion1	estacion2	distancia
1	1	0
1	2	97.07
1	3	44.45
1	4	119.1
1	5	127.2
1	6	109.56
1	7	86.34
1	8	109.66
1	9	188.05
1	10	156.8
2	1	97.07
2	2	0
2	3	128.22
2	4	165.87
2	5	224.27
2	6	132.58
2	7	154.07
2	8	58.32
2	9	165.92
2	10	59.77
3	1	44.45
3	2	128.22
3	3	0
3	4	81.52
3	5	107.58
3	6	150.94
3	7	115.23

Estacion

id	nombre	municipio_id	latitud	longitud
1	Valladolid Campo Grande	1	41.642	-4.727
2	Segovia Guiomar	2	40.91	-4.094
3	Palencia	3	42.015	-4.534
4	Burgos Rosa Manzano	4	42.367	-3.666
5	León	5	42.594	-5.582
6	Salamanca	6	40.958	-5.671
7	Zamora	7	41.515	-5.751
8	Ávila	8	40.656	-4.7
9	Soria	9	41.768	-2.468
10	Madrid Chamartín-Clara Campoamor	10	40.4722	-3.6826

Municipio

id	nombre	n_habitantes	latitud	longitud	provincia	ccaa
1	Valladolid	295639	41.6523	-4.7245	Valladolid	Castilla y León
2	Segovia	51378	40.9429	-4.1088	Segovia	Castilla y León
3	Palencia	76438	42.0095	-4.528	Palencia	Castilla y León
4	Burgos	175821	42.3439	-3.6969	Burgos	Castilla y León
5	León	121281	42.5987	-5.5671	León	Castilla y León
6	Salamanca	144436	40.9701	-5.6635	Salamanca	Castilla y León
7	Zamora	59475	41.5033	-5.7446	Zamora	Castilla y León
8	Ávila	58085	40.6566	-4.6819	Ávila	Castilla y León
9	Soria	39821	41.7633	-2.4688	Soria	Castilla y León
10	Madrid	3334730	40.4168	-3.7038	Madrid	Comunidad de Madrid

Parada

ruta	estacion	n_secuencia	km_origen
1	1	1	0
1	10	2	193
2	10	1	0
2	1	2	193
3	1	1	0
3	2	2	108
4	2	1	0
4	10	2	92
5	1	1	0
5	3	2	50
6	3	1	0
6	5	2	127
7	6	1	0
7	7	2	66
8	8	1	0
8	10	2	115
9	4	1	0
9	3	2	90
9	1	3	140
10	9	1	0
10	10	2	231
11	5	1	0
11	3	2	127
12	7	1	0
12	6	2	66
13	10	1	0
13	8	2	115
14	1	1	0
14	3	2	50
14	4	3	140
15	10	1	0
15	9	2	231

Viaje

id	ruta	fecha	horario
1	1	2026-03-24	08:30:00
2	1	2026-03-24	14:00:00
3	1	2026-03-25	18:30:00
4	2	2026-03-24	10:00:00
5	2	2026-03-25	19:00:00
6	3	2026-03-24	09:15:00
7	3	2026-03-26	17:10:00
8	4	2026-03-24	11:20:00
9	4	2026-03-25	16:40:00
10	4	2026-03-26	20:10:00
11	5	2026-03-24	07:45:00
12	5	2026-03-25	15:30:00
13	5	2026-03-26	19:20:00
14	6	2026-03-24	13:10:00
15	7	2026-03-24	18:00:00
16	7	2026-03-26	12:00:00
17	8	2026-03-24	06:50:00
18	8	2026-03-25	16:10:00
19	8	2026-03-26	21:00:00
20	9	2026-03-25	16:30:00
21	9	2026-03-26	09:40:00
22	10	2026-03-25	17:45:00
23	10	2026-03-26	07:30:00
24	11	2026-03-24	12:15:00
25	12	2026-03-26	07:55:00
26	13	2026-03-25	09:05:00
27	13	2026-03-26	13:25:00
28	14	2026-03-25	11:50:00
29	14	2026-03-26	18:05:00
30	15	2026-03-25	15:15:00

2 Esquemas Origen

En este documento se muestran los esquemas de las fuentes de datos origen de nuestro sistema integrador Tren-ES. Las fuentes de datos son las siguientes:

- Wikidata. Para obtener información de los municipios, como su nombre, código del INE, coordenadas geográficas o la población. Para extraer estos datos utilizaremos un *punto sparql*
- INE. Para conectar los códigos de los municipios con las provincias y comunidades autónomas. Para extraer estos datos utilizaremos la *API oficial*.
- Renfe. Dos distintas:
  - Renfe Data. Obtenemos información a partir de la *API de Renfe* de todas las estaciones que existen en España.
  - Renfe horarios. Un pequeño formulario desde el que se puede obtener la información de todos los horarios de tramos (Estación Origen, Estación Destino) disponibles en la web de renfe y las rutas correspondientes. Para extraer estos datus utilizaremos *web scraping*.
- ADIF. Información en tiempo real de las salidas y llegadas en cada estación. Nos permite contrastar los horarios planificados con los reales. Para extraer estos datus utilizaremos *web scraping*.

2.1 Forma conjuntiva de las tablas origen

- wikidata_MUNICIPIO(codigo INTEGER, label VARCHAR(255), poblacion INTEGER, coordenadas POINT) Tabla de datos extraidos de Wikidata. La tabla está formada por un campo `codigo` que contiene el identificador INE del municipio, un campo `label` con el nombre del municipio en texto legible, la columna `poblacion` que almacena el número de habitantes de ese municipio y las `coordenadas`, una tupla de tipo Point que almacena las coordenadas geográficas de dicho municipio.
- INE_MUNICIPIO(cmun INTEGER, cpro INTEGER, dc INTEGER, nombre VARCHAR(100)) Datos oficiales sobre municipios ofrecidos por el INE. Formados por un código de municipio `cmun`, que identifica a un municipio *dentro* de una provincia, el código de la provincia a la que pertenece, `cpro`, un dígito de control (que no nos genera interés) `dc` y el nombre en texto legible, `nombre`.
- INE_PROVINCIA(cpro INTEGER, nombre VARCHAR(100), codauto INTEGER, ccaa VARCHAR(100)) Datos oficiales sobre provincias ofrecidos por el INE. Es una tabla que cruza provincias con su respectiva comunidad autónoma. Formada por la información de provincia, `cpro` y `nombre`, código de la provincia y nombre en texto legible respectivamente, e información sobre la Comunidad Autónoma a la que pertenece, `codauto` y `ccaa`, código de la Comunidad Autónoma y nombre en texto legible respectivamente.
- data_renfe_ESTACION(id INTEGER, codigo INTEGER, descripcion VARCHAR(255), latitud NUMERIC(9,6), longitud NUMERIC(9,6), direccion VARCHAR(255), c_p VARCHAR(5), poblacion VARCHAR(100), provincia VARCHAR(100), pais VARCHAR(100)) Este esquema, se ofrece en la sección de datos abiertos de Renfe. Contiene información sobre todas las estaciones de trenes que hay en España. Se compone por, un `id` de asignación secuencial, un código de estación `codigo`, el nombre en texto legible guardado en `descripción`, las coordenadas geográficas por separado en `latitud` y `longitud`, una dirección guardada en la columna `direccion` y el código postal asociado `c_p` y finalmente datos territoriales de la ubicación, `población`, `provincia` y `pais`.
- horarios_renfe_HORARIO(recorrido VARCHAR(255), salida TIME, llegada TIME, duracion INTERVAL) Datos extraídos de un formulario de la página de Renfe. El horario son viajes que van desde una estación de origen hasta una de destino. `recorrido` es la ruta que realiza ese viaje, `salida` y `llegada` son las estaciones que une dicho viaje (y que son paradas de la ruta). Duración es el tiempo en horas y minutos que tarda en completarse el viaje.
- horarios_renfe_RUTA(codigo INTEGER, tipo VARCHAR(20), paradas TEXT) Se extraen del mismo formulario, contiene información sobre una ruta en específico. Las rutas son conjuntos de estaciones que se recorren en un orden específico. Una ruta tiene un `codigo` que la identifica frente al resto, y un `tipo` de tren que la recorre (MD, AV, AVLO, ...) y `paradas` representa la lista de paradas. Sabemos que no es una representación normalizada, pero nos parecía que añadía mucha complejidad a representar los orígenes, y por motivos de simpleza lo hemos hecho así. En el esquema mediador se trata y soluciona este problema.
- adif_SALIDAS(hora TIME, destino VARCHAR(255), tren VARCHAR(100), via INTEGER, estacion VARCHAR(255)) Información accesible desde las páginas de datos en tiempo real sobre las estaciones españolas. Contiene información sobre las salidas en tiempo real. `hora` es el cuándo sale el tren de la estación, el campo `destino` contiene la estación destino de dicha salida, `tren` almacena tanto la ruta de dicho viaje que sale como el tipo de tren que la atiende, una columna `via` desde dónde es dicha salida y `estacion` la estación de la que provienen los datos.

- adif_LLEGADAS(hora TIME, origen VARCHAR(255), tren VARCHAR(100), via INTEGER, estacion VARCHAR(255)) La entidad de llegadas contiene la misma información que salidas, pero intercambiando cualquier dato por los correspondientes a las llegadas.

wikidata_MUNICIPIO		
INTEGER	CODIGO	CÓDIGO
VARCHAR	LABEL	
INTEGER	POBLACION	POBLACIÓN
POINT	COORDENADAS	

INE_MUNICIPIO		
INTEGER	CMUN	CÓDIGO DE MUNICIPIO
INTEGER	CPRO	CÓDIGO DE PROVINCIA
INTEGER	DC	DÍGITO DE CONTROL
VARCHAR	NOMBRE	

data_renfe_ESTACION		
INTEGER	ID	
INTEGER	CODIGO	CÓDIGO
VARCHAR	DESCRIPCION	NOMBRE ESTACIÓN
NUMERIC	LATITUD	
NUMERIC	LONGITUD	
VARCHAR	DIRECCION	DIRECCIÓN
VARCHAR	C_P	C.P.
VARCHAR	POBLACION	
VARCHAR	PROVINCIA	
VARCHAR	PAIS	

INE_PROVINCIA		
INTEGER	CPRO	CÓDIGO DE PROVINCIA
VARCHAR	NOMBRE	NOMBRE DE PROVINCIA
INTEGER	CODAUTO	CÓDIGO DE CCAA
VARCHAR	CCAA	NOMBRE COMUNIDAD AUTÓNOMA

horarios_renfe_HORARIO		
VARCHAR	RECORRIDO	TREN/RECORRIDO
TIME	SALIDA	
TIME	LLEGADA	
TIME	DURACION	HORAS/MINUTOS

horarios_renfe_RUTA		
INTEGER	CODIGO	CÓDIGO DE RUTA
VARCHAR	TIPO	TIPO DE TREN
TEXT	PARADAS	NOMBRE ESTACIONES

adif_SALIDAS	
TIME	HORA
VARCHAR	DESTINO
VARCHAR	TREN
INTEGER	VIA
VARCHAR	ESTACION

adif_LLEGADAS	
TIME	HORA
VARCHAR	ORIGEN
VARCHAR	TREN
INTEGER	VIA
VARCHAR	ESTACION

A partir del mediador y los esquemas de las fuentes de datos podemos acabar con los siguientes esquemas GAV:

2.1.1 Municipio

Es necesario cruzar los datos de los datos de *wikidata* y los del *INE* para poder obtener de forma correcta los códigos de provincia y comunidad autónoma.

`Municipio(id, nombre, n_habitantes, latitud_m, longitud_m, provincia, CCAA)` $\supseteq$

`wikidata_MUNICIPIO(id, nombre ,n_habitantes, coordenadas), latitud_m=latitud(coordenadas), longitud_m=longitud(coordenadas), INE_MUNICIPIO(cmun, cpro, v1, v2), id==cmun+cpro, INE_PROVINCIA(cpro, provincia, v3, CCAA)`

2.1.2 Estacion

Hay que tener en cuenta que solo debemos obtener estaciones de España.

`Estacion(id, municipio, nombre, latitud_e, longitud_e)` $\supseteq$

`data_renfe_ESTACION(v1, id, nombre, latitud_e, longitud_e, v2, v3, nombre_mun, v4, v5), wikidata_MUNICIPIO(municipio, nombre_mun ,v6, pais), pais==España`

2.1.3 Distancia

En este caso solo guardamos las distancias entre las estaciones de forma unidireccional, ya que la distancia entre una estación y otra es la misma que entre la segunda y la primera, y así evitamos información repetida.

`Distancia(estacion1, estacion2, distancia)` $\supseteq$

`data_renfe_ESTACION(v1, estacion1, v2, latitud_1, longitud_1, v3, v4, v5, v6, pais), data_renfe_ESTACION(v8, estacion2, v9, latitud_2, longitud_2, v10, v11, v12, v13, pais), distancia=dist(latitud_1, latitud_2, longitud_1, longitud_2), estacion1 < estacion2, pais==España`

2.1.4 Ruta y Parada

En las fuentes de datos obtenemos desde *horarios_renfe_RUTA* una lista de paradas. Para poder extraer cada parada por separado debemos normalizar esta lista, por lo que nos ayudamos de una vista auxiliar. Esta

vista no se encuentra en los archivos SQL en este hito.

2.1.4.1 Vista auxiliar: Parada_aux Esta vista auxiliar no se encuentra en el esquema mediador, utilizaremos una función para normalizar la lista y poder extraer las paradas:

`Parada_aux(codigo_ruta, num_secuencia, nombre_parada)`$\supseteq$

```
horarios_renfe_RUTA(lista_paradas, codigo_ruta),
(num_secuencia, nombre_parada) =unlist(lista_paradas)
```

A partir de esta vista auxiliar podemos definir correctamente parada:

`Parada(ruta, estacion, num_secuencia, km_origen)` $\supseteq$

```
Parada_aux(ruta, num_secuencia, nombre_parada),
data_renfe_ESTACION(v1, estacion, nombre_parada, v3, v4, v5, v6, v7, v8, pais), pais==España
```

Para definir la ruta volvemos a necesitar funciones externas. Una ruta se define por un identificador propio y una tupla origen y destino de estaciones de la ruta. Para esto debemos obtener los números de secuencia mínimo y máximo de cada entrada de *Parada_aux*.

`Ruta(id, origen, destino, tipo)` $\supseteq$

```
Parada_aux(id, num_secuencia, nombre_parada), nombre_origen=getMinSecuencia(num_secuencia),
nombre_destino=getMaxSecuencia(num_secuencia), data_renfe_ESTACION(v1, origen, nombre_origen,
v3, v4, v5, v6, v7, v8, pais), data_renfe_ESTACION(v10, destino, nombre_destino, v11, v12,
v13, v14, v15, v16, pais), pais==España
```

2.1.5 Viaje

El horario se refiere a la hora de llegada en la estación destino del viaje específico, recogido de la web de *ADIF*.

`Viaje(id, ruta, fecha, horario)` $\supseteq$

```
Parada_aux(ruta, num_secuencia, nombre_parada), nombre_origen=getMinSecuencia(num_secuencia),
data_renfe_ESTACION(v1, estacion, nombre_origen, v3, v4, v5, v6, v7, v8, pais),
adif_SALIDAS(fecha, horario, v10, v11, estacion), id=secuencia(), pais==España
```

No se han considerado simplificaciones para ninguna tabla. Para la tabla Municipio no se puede simplificar ya que es necesario hacer el cruce de tablas con el INE para obtener correctamente el código de provincia y Comunidad Autónoma. Para el resto de tablas no vemos simplificación posible.

A partir del esquema mediador y los esquemas de las fuentes de datos, podemos obtener los siguientes esquemas LAV:

2.1.6 wikidata_MUNICIPIO

Necesitamos concatenar las coordenadas de latitud y longitud extraídas en los esquemas GAV

`wikidata_MUNICIPIO(codigo, label, poblacion, coordenadas)`$\subseteq$

```
Municipio(codigo, label, poblacion, latitud, longitud, v1, v2),
coordenadas = coords(latitud, longitud)
```

2.1.7 INE_MUNICIPIO

Como no extraemos el dígito de control dc, no podemos obtenerlo de los esquemas del mediador.

`INE_MUNICIPIO(cmun, cpro, dc, nombre)`$\subseteq$

```
Municipio(id, nombre, v1, v2, v3, v4, v5), id == cmun + cpro
```

2.1.8 INE_PROVINCIA

Como no extraemos el código de CCAA, no podemos obtenerlo de los esquemas del mediador.

`INE_PROVINCIA(cpro, nombre, codauto, ccaa)`$\subseteq$

`Municipio(id, nombre, v1, v2, v3, v4, v5), id == cmun + cpro`

2.1.9 data_renfe_ESTACION

Como estación venía de un formulario con varios campos que no extraemos, no podemos reconstruir exactamente la fuente completa a partir de nuestros esquemas.

`data_renfe_ESTACION(v1, id, descripcion, latitud, longitud, v2, c_p, poblacion, provincia, pais)`$\subseteq$

`Estacion(id, id_mun, descripcion, latitud, longitud),`
`Municipio(id_mun, poblacion, v3, v4, v5, provincia, v6)`

2.1.10 horarios_renfe_RUTA

Al contrario que en los esquemas GAV, debemos crear una lista a partir de nuestros esquemas.

`horarios_renfe_RUTA(codigo, paradas)`$\subseteq$

`Parada(codigo, estacion, n_secuencia, v4), paradas = list(estacion, n_secuencia)`

2.1.11 adif_SALIDAS

No podemos conseguir los campos tren y via desde nuestros esquemas, por lo que la tabla no se puede construir por completo.

`adif_SALIDAS(hora, destino, tren, via, estacion)`$\subseteq$

`Viaje(id_viaje, ruta, fecha, hora), Ruta(ruta, origen, id_estacion_destino, tipo),`
`Estacion(id_estacion_destino, estacion, destino, v2, v3)`

Al igual que en los esquemas GAV, no se ha considerado ninguna simplificación.

3 Reformulación GAV y LAV de las Consultas

En esta sección vamos a hacer la reformulación GAV y LAV de las consultas. Para la reformulación GAV emplearemos la descomposición de consultas y para la reformulación LAV utilizaremos el algoritmo de los *Buckets*.

3.1 Consulta 1

- Estaciones de tren en poblaciones de menos de 10000 habitantes.

La consulta se puede escribir sobre el esquema mediador como:

`Q1(id, nombre_estacion, nombre_municipio):- Estacion(id, id_mun, nombre_estacion, lat_e, lon_e), Municipio(nombre_municipio, habitantes, id_mun, lat_m, lon_m, prov, ccaa), hab<10000`

3.1.1 Reformulación GAV

Para hacer la reformulación GAV basta con sustituir los átomos en la consulta global por sus respectivas definiciones en función de las fuentes, a partir de la formulación GAV de las fuentes vistas en apartados anteriores.

Para el átomo *Estacion*, sustituimos por su correspondiente formulación GAV: `Estacion(id, municipio, nombre, latitud_e, longitud_e)` $\subseteq$

`data_renfe_ESTACION(v1, id, nombre, latitud_e, longitud_e, v2, v3, nombre_mun, v4, v5), wikidata_MUNICIPIO(municipio, nombre_mun ,v6, pais), pais==España`

E igualmente con *Municipio*, que queda: `Municipio(id, nombre, n_habitantes, latitud_m, longitud_m, provincia, CCAA)` $\subseteq$

`wikidata_MUNICIPIO(id, nombre ,n_habitantes, coordenadas), latitud_m=latitud(coordenadas), longitud_m=longitud(coordenadas), INE_MUNICIPIO(cmun, cpro, v1, v2), id==cmun+cpro, INE_PROVINCIA(cpro, provincia, v3, CCAA)`

Incluimos dichas formulaciones en la consulta global, añadiendo la restricción de habitantes, que da como resultado:

```
Q1'(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, v5), wikidata_MUNICIPIO(id_mun, nombre_municipio, v6, v7),
wikidata_MUNICIPIO(id_mun, nombre_municipio, hab, coord), lat_m=latitud(coord),
lon_m=longitud(coord), INE_MUNICIPIO(cmun, cpro, v8, v9),
INE_PROVINCIA(cpro, prov, v10, ccaa),
id_mun=cmun+cpro, hab<10000
```

Por último, podemos ver que el átomo *wikidata_MUNICIPIO* se encuentra repetido, por lo que podríamos pensar en simplificar para eliminar redundancias. Podríamos hacer mappings de contención para ver equivalencias, pero es evidente que, dado que v6 y v7 no se utilizan, podemos eliminar el primer átomo, quedando la consulta final como:

```
Q1'(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, pais), wikidata_MUNICIPIO(id_mun, nombre_municipio,
hab, coord), lat_m=latitud(coord), lon_m=longitud(coord),
INE_MUNICIPIO(cmun, cpro, v8, v9), INE_PROVINCIA(cpro, prov, v10, ccaa),
id_mun=cmun+cpro, hab<10000, pais=España
```

3.1.2 Reformulación LAV

Para hacer la reformulación LAV debemos utilizar el algoritmo basado en buckets.

1. Creamos y llenamos los buckets: como hay dos átomos en la consulta, creamos dos buckets. Para que una vista forme parte de un bucket es necesario que cumpla tres condiciones:
   a. Un átomo de la vista y el átomo g para el cual construimos el bucket afectan a la misma relación.
   b. Los predicados (condiciones) de la vista y los de la consulta Q son mutuamente satisfacibles.
   c. Si en g aparece alguna variable cabecera de la consulta Q, entonces también debe aparecer en la cabecera de la vista.

Siguiendo las normas de construcción de buckets, para el primer átomo, su bucket estará compuesto únicamente por *wikidata_ESTACION*, que se añade al bucket. El segundo bucket estará compuesto únicamente por la fuente *wikidata_MUNICIPIO*, que se añade al segundo bucket.

2. Reformulación como producto cartesiano: hacemos el producto cartesiano entre los buckets. Como solo hay una posibilidad por bucket solo podremos hacer una reformulación. Si nos fijamos, esta reformulación será la misma que la reformulación GAV:

```
Q1''(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, v5), wikidata_MUNICIPIO(id_mun, nombre_municipio, v6, v7),
wikidata_MUNICIPIO(id_mun, nombre_municipio, hab, coord), lat_m=latitud(coord),
lon_m=longitud(coord), INE_MUNICIPIO(cmun, cpro, v8, v9),
INE_PROVINCIA(cpro, prov, v10, ccaa), id_mun=cmun+cpro, hab<10000
```

3. Posible simplificación: como ya vimos anteriormente, es posible simplificar la consulta, por lo que el resultado final es:

```
Q1'(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, v5), wikidata_MUNICIPIO(id_mun, nombre_municipio, hab,
coord), lat_m=latitud(coord), lon_m=longitud(coord), INE_MUNICIPIO(cmun, cpro, v8, v9),
INE_PROVINCIA(cpro, prov, v10, ccaa), id_mun=cmun+cpro, hab<10000
```

Como ya hemos visto, la reformulación LAV y GAV coinciden aunque hayamos utilizado diferentes algoritmos para su construcción. Es por esto que para las siguientes dos consultas únicamente se dejará indicada la reformulación final.

3.2 Consulta 2

Esta consulta tiene un `OR` para poder trabajar con la tabla distancia. Es por ello que la consulta se define como:

$$Q2 = Q2_a \cup Q2_b$$

Estas subconsultas se pueden escribir sobre el esquema mediador como:

```
Q2a(id):- Viaje(id, ruta, v1, v2), Ruta(ruta, origen, destino, v3),
Distancia(origen, destino, dist), dist<30
```

```
Q2a(id):- Viaje(id, ruta, v1, v2), Ruta(ruta, origen, destino, v3),
Distancia(destino, origen, dist), dist<30
```

3.2.1 Reformulación GAV/LAV

La consulta en forma GAV queda extremadamente larga, puesto que para calcular distancias hay que cruzar datos con la vista auxiliar

```
Q2a'(id):- Parada_aux(ruta, num_secuencia, nombre_parada),
nombre_origen=getMinSecuencia(num_secuencia), data_renfe_ESTACION(v1, estacion, nombre_origen,
v3, v4, v5, v6, v7, v8, pais), adif_SALIDAS(fecha, horario, v10, v11, estacion),
id=secuencia(), pais==España, Parada_aux(ruta, num_secuencia, nombre_parada),
nombre_origen=getMinSecuencia(num_secuencia), nombre_destino=getMaxSecuencia(num_secuencia),
data_renfe_ESTACION(v1, origen, nombre_origen, v12, v13, v14, v15, v16, v17, pais),
data_renfe_ESTACION(v36, destino, nombre_destino, v18, v19, v20, v21, v22, v23, pais),
pais==España, data_renfe_ESTACION(v24, origen, v25, latitud_1, longitud_1, v26, v27,
v28, v29, pais), data_renfe_ESTACION(v30, destino, v31, latitud_2, longitud_2, v32,
v33, v34, v35, pais), distancia=dist(latitud_1, latitud_2, longitud_1, longitud_2),
origen < destino, pais==España, dist<30
```

Tras simplificar, la consulta queda como:

```
Q2a'(id):- Parada_aux(ruta, num_secuencia, nombre_parada),
nombre_origen=getMinSecuencia(num_secuencia), nombre_destino=getMaxSecuencia(num_secuencia),
data_renfe_ESTACION(v1, origen, nombre_origen, latitud_1, longitud_1, v2, v3, v4, v5, v6),
data_renfe_ESTACION(v7, destino, nombre_destino, latitud_2, longitud_2, v8, v9, v10, v11,
v12), adif_SALIDAS(fecha, horario, v13, v14, origen), id=secuencia(),
distancia=dist(latitud_1, latitud_2, longitud_1, longitud_2),
pais==España, origen < destino, distancia<30
```

Q2b' cubrirá aquellos casos en los que destino $<$ origen.

```
Q2a'(id):- Parada_aux(ruta, num_secuencia, nombre_parada),
nombre_origen=getMinSecuencia(num_secuencia), nombre_destino=getMaxSecuencia(num_secuencia),
data_renfe_ESTACION(v1, origen, nombre_origen, latitud_1, longitud_1, v2, v3, v4, v5, v6),
```

```
data_renfe_ESTACION(v7, destino, nombre_destino, latitud_2, longitud_2, v8, v9, v10, v11,
v12), adif_SALIDAS(fecha, horario, v13, v14, origen), id=secuencia(),
distancia=dist(latitud_1, latitud_2, longitud_1, longitud_2),
pais==España, destino < origen, distancia<30
```

3.3 Consulta 3

Si vemos la definición de la consulta 3, podemos ver que para poder obtener las estaciones con 0 viajes entre sí debemos hacer un `LEFT JOIN` sobre el esquema mediador, y no sabemos como poner dicha operación en forma conjuntiva. Por tanto, tampoco sabríamos escribir la consulta con la reformulación GAV o la reformulación LAV.

4 Scraping

4.1 ADIF

Desde la página web de consulta de horarios de estaciones en tiempo real deADIF hemos extraido mediante scrapping las tablas de las fuentes adif_SALIDAS y adif_LLEGADAS. No ha sido sencillo, pues es una aplicación web relativamente moderna y está bien preparada frente a interacción atomatizada, lo cuál ha resultado un reto. La implementación del scraping se puede ver en el archivo 'scraping_adif.py'. Este fichero actua a modo demo, por lo que no hay ningún tipo de interacción con el usuario. Simplemente accede a una página (la estación de Valladolid) y extrae las dos tablas. A continuación se procede a explicar la lógica y detalles técnicos:

Una petición http básica no era suficiente para poder acceder al contenido real, pues la aplicación respondía con un html especial indicando que el acceso no estaba permitido:

```
<HTML>
<HEAD>
<TITLE>Access Denied</TITLE>
</HEAD>
<BODY>
<H1>Access Denied</H1>

You don't have permission to access
</HTML>
```

por lo que tuvimos que rechazar una primera aproximación usando los módulos `requests` con `BeautifullSoup`. Por tanto, probamos a usar el módulo `playwright`, desarrollado por Microsoft y mucho más potente.

El módulo proporciona una interfáz muy sencilla para manejar versiones "headless" de un navegador, con el objetivo de simular la navegación de un usuario real, pudiendo interactuar con elementos de la aplicación sin simplemente limitarse a la respuesta de una petición básica. Aún así, el sistema de detección de "bots" de la web seguía respondiendo con ficheros html de rechazo, pues parece ser que también era capaz de identificar que se trataba de un navegador "headless". Pudimos solucionar este problema creando un nuevo "contexto" de navegador que simulase una ventana gráfica, pero sin que de verdad esta existiese; gracias a esto hemos sido capaces de acceder a la información.

El flujo del programa es muy simple:

1. Se crea una "ventana" virtual y una página sobre esta, posteriormente se navega a la ruta (para el caso de valladolid, https://www.adif.es/w/10600-valladolid-c.-g.) y se espera a que todo el DOM esté en estado *attached* para asegurar que los elementos a los que queremos acceder están disponibles en el momento de buscarlos.
2. Dependiendo de si se quieren estraer las tablas de llegadas o salidas, hay que pulsar un botón de "salidas" o no (la opción de "llegadas" parece que viene seleccionada por defecto). Este es el único paso diferente, a partir de aquí el programa es igual para ambos salvo alguna pequeña variante de notación. Para

acceder a la tabla hay que filtrar por el identificador `#horas-trenes-estacion-(salidas|llegadas)` dependiendo de cual sea la fuente que se quiera extraer.

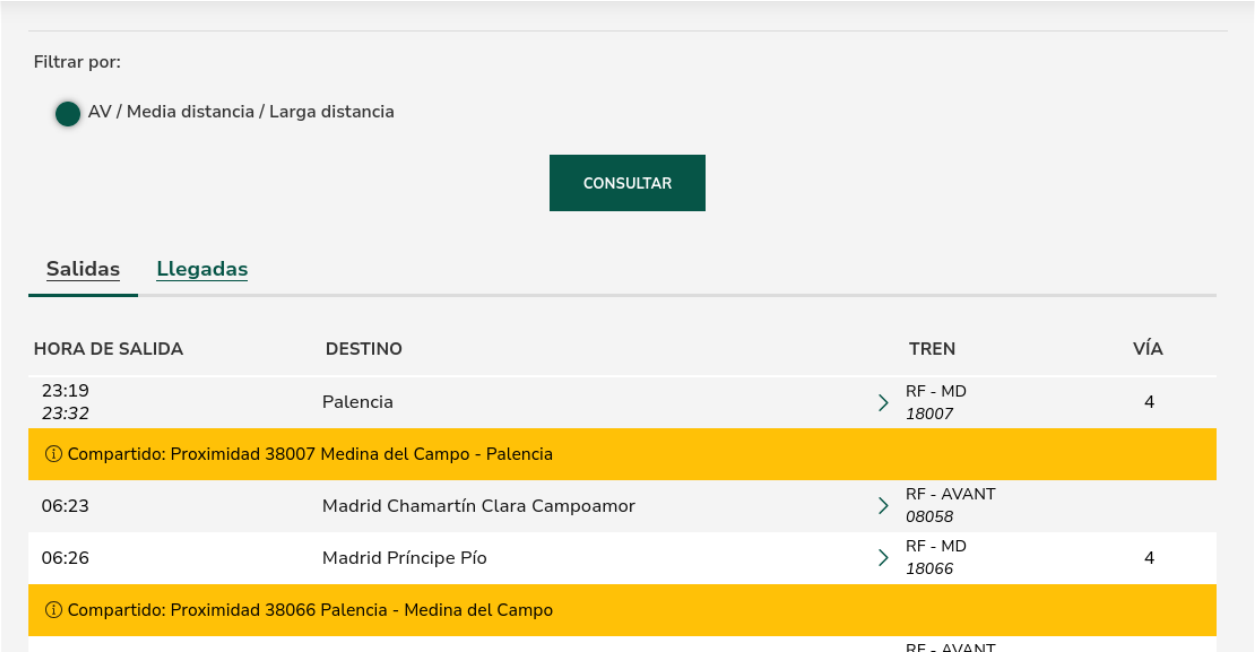


Figure 2: Captura de pantalla de la página de ADIF

3. La extracción de información es la misma, pues usa los mismos identificadores para las columnas, incluso la estación de origen o destino también tiene el mismo identificador `col-destino` independientemente de si se habla de salidas o llegadas. Al ser una tabla, extraer los datos es tan sencillo como iterar sobre todos los objetos de tipo `tr` y acceder a cada elemento `td` por columna. Una cosa importante y que tuvimos que solucionar fue que hay dos tipos de "filas", las que contienen la información que nos interesa y las amarillas, que proporcionan información sobre problemas en la infrastructura, que no tienen el mismo formato y por tanto pueden producir errores si no se trata bien.
4. Finalmente, se "serializan" los datos extraidos en un csv usando la función de la librería estandar `csv.DictWritter`.

4.2 RENFE

Hemos utilizado scraping sobre la web de renfe horarios. Para conseguir esta información hemos tenido que rellenar un formulario utilizando *playwright* y *BeautifulSoup* en Python para acceder, ya que con la librería *requests* aparecían problemas para realizar el scraping.

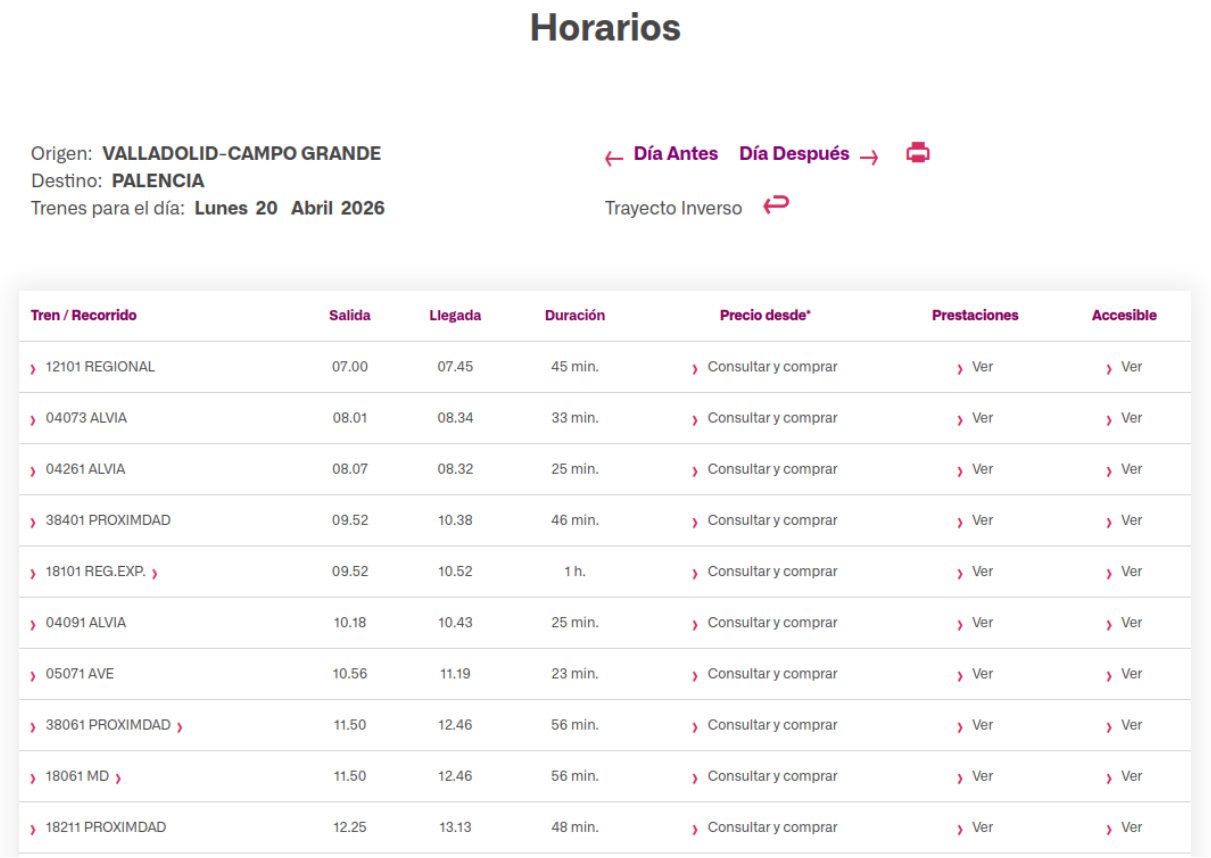
El scraping realizado se puede ver en el archivo *scraping_renfe_horarios.py*, y el flujo del programa es el siguiente:

1. Se introduce al programa la información relativa al cuestionario con el siguiente formato: [cod_estacion_origen]-[cod_estacion_destino]-[dia]-[mes]-[año], de tal forma que para obtener los recorridos entre Valladolid Campo Grande (estación número 10600) y Palencia (estación número 14100) para el día 20 de abril de 2026 podríamos ejecutar el programa como:

```
{bash} $ echo "10600-14100-20-04-2026" | python3 scraping_renfe_horarios.py
```

Como se puede ver, el programa solo hace scraping sobre dos estaciones para un día concreto. El motivo de esto es que todavía no heos obtenido el listado de todas las estaciones para poder hacer el scraping de todos los viajes, aunque el programa está preparado para soportar varias llamadas seguidas.

2. Se accede a la web, accediendo al *iframe* que contiene el formulario. Si no se accede a este iframe no se pueden obtener los elementos html del formulario. Cuando se accede, se rellenan los datos acorde a la entrada del programa y se hace click en el botón **BUSCAR**.
3. El *iframe* cambia y muestra una tabla como la de la imagen de debajo. Accedemos a cada linea de la tabla y obtenemos el primer valor, que redirige a otra página web mediante una función javaScript y un hiperenlace.



Horarios

Origen: **VALLADOLID-CAMPO GRANDE**
Destino: **PALENCIA**
Trenes para el día: **Lunes 20 Abril 2026**

← **Día Antes** **Día Después** →

Trayecto Inverso

Tren / Recorrido	Salida	Llegada	Duración	Precio desde*	Prestaciones	Accesible
› 12101 REGIONAL	07.00	07.45	45 min.	› Consultar y comprar	› Ver	› Ver
› 04073 ALVIA	08.01	08.34	33 min.	› Consultar y comprar	› Ver	› Ver
› 04261 ALVIA	08.07	08.32	25 min.	› Consultar y comprar	› Ver	› Ver
› 38401 PROXIMDAD	09.52	10.38	46 min.	› Consultar y comprar	› Ver	› Ver
› 18101 REG.EXP. ›	09.52	10.52	1 h.	› Consultar y comprar	› Ver	› Ver
› 04091 ALVIA	10.18	10.43	25 min.	› Consultar y comprar	› Ver	› Ver
› 05071 AVE	10.56	11.19	23 min.	› Consultar y comprar	› Ver	› Ver
› 38061 PROXIMDAD ›	11.50	12.46	56 min.	› Consultar y comprar	› Ver	› Ver
› 18061 MD ›	11.50	12.46	56 min.	› Consultar y comprar	› Ver	› Ver
› 18211 PROXIMDAD	12.25	13.13	48 min.	› Consultar y comprar	› Ver	› Ver

Figure 3: Tabla de horarios RENFE

4. Formateamos el hiperenlace para hacerlo coincidir con el correcto. Esto equivale a eliminar saltos de línea y sustituir espacios en blanco por *%20*, de tal forma que la url:

`horarios.renfe.com/HIRRenfeWeb/recorrido.do?O=10600&D=14100&F=2026-04-20&T=04073&G=1&TT=ALVIA%20%20%20%2`

Equivale a la imagen de debajo.

5. Accedemos a estas url (una por ruta) y obtenemos el listado de paradas
6. Accedemos a dos archivos csv, correspondientes a los Esquemas en Origen de *horarios_renfe_RUTA* y *horarios_renfe_HORARIO*, y rellenamos los datos. Accedemos a estos archivos en modo append: si no existen se crean y si existen se añade la nueva información al final (así podemos crear muchas llamadas al programa para diferentes rutas).

Por último, se ha añadido la opción *–verbose* para obtener logs del programa.

5 SPARQL

5.1 WikiData

Los datos de los municipios de España los vamos a coger de WikiData, mediante un script de *Python* que utilice el servicio de consultas del propio WikiData.

En este caso, la consulta ha sido directa y sencilla, necesitando únicamente el nombre del municipio, la población que tiene y sus coordenadas, en formato POINT. Únicamente hay que tener en cuenta que WikiData puede cancelar la consulta si no se especifica un User-Agent (debido a su política de Robots).

Columna	Tipo
CODIGO	text
LABEL	text
POOBLACION	numeric
COORDENADAS	Point

Para lograrlo, hemos hecho la siguiente consulta:

```
SELECT ?codigo ?label ?poblacion ?coordenadas WHERE {
  ?municipio (wdt:P31/(wdt:P279*)) wd:Q2074737;
    wdt:P772 ?codigo;
    wdt:P1082 ?poblacion;
    wdt:P625 ?coordenadas.

  SERVICE wikibase:label {
    bd:serviceParam wikibase:language "es".
    ?municipio rdfs:label ?label.
  }
}
ORDER BY ?codigo
```